

CSE 470 — Midterm Spring 2024 (Set A): Model Solutions

BRAC University · Duration 1 hr 15 min · Full marks 25 | Q1 = 7 · Q2 = 6 · Q3 = 5 · Q4 = 7

Q1 — Dunki Route App 7 marks

A. Which methodology are they following? 1

Agile — specifically the SCRUM framework.

B. Detailed implementation process of the chosen methodology 4

Scrum is the most widely used Agile method for project management. The full software is delivered in **1-4 week iterations (sprints)**. It is implemented through three **Roles**, three **Ceremonies** and three **Artifacts**.

Element	How it works in Scrum	Where it appears in the scenario
ROLES		
Product Owner	Decides features, release date, prioritisation and budget; conveys a clear vision to the team and answers queries.	The agent who provided the instructions on how to reach England and set the plan.
Scrum Master	Enacts Scrum values and practices; removes impediments and keeps everyone productive.	Whoever among them kept the journey on track and cleared obstacles.
Team	5-10 members, self-organizing and cross-functional; membership changes only between sprints.	Hardy, Manu and Buggu — a small self-organizing team.
CEREMONIES		
Sprint Planning	The team selects the highest-priority items from the Product Backlog into the Sprint Backlog for the coming sprint.	"After discussing the plan, they prioritized the route " — Pakistan → Afghanistan → Iran → Turkey → England.
Daily Scrum Meeting	Daily, ~15 minutes, stand-up. Each member answers: <i>What did you do yesterday? What will you do today? What obstacles are in your way?</i> It is not for problem solving .	" Each evening, they briefly discussed progress, estimated travel times, and anticipated challenges for the next day. "
Sprint Review & Retrospective	Review = the team demonstrates what it accomplished; informal, whole team participates. Retrospective = identify scopes of improvement, what worked well and what went wrong.	" Celebrated reaching each destination with a party to share achievements. "
ARTIFACTS		
Product Backlog	The requirements — a list of all desired work, prioritised by the Product Owner and reprioritised at the start of each sprint.	"They documented this plan in their to-do list ."
Sprint Backlog	High-priority items selected for the current sprint; remaining work updated daily; cannot be added to, deleted or changed mid-sprint.	"Starting in Pakistan, they crafted a new checklist to anticipate potential obstacles."
Burndown Chart	A display of remaining work vs time, updated every day.	"They tracked progress with a map ."

The cycle repeats — each leg of the journey (Pakistan → Afghanistan, truck through Iran, train across Turkey, ship to England) is one sprint, ending in a review/celebration, and this continued until they reached England, i.e. until the product was complete.

C. Two functional and two non-functional requirements 2

Functional Requirements

1. The system shall allow travelers to visualize their route on a map and receive shortest-path guidance.
2. The system shall allow travelers to track their journey progress through AI and shall alert them of potential dangers.

Non-Functional Requirements

1. The system shall ensure smooth performance under all conditions without degradation.
2. The system shall encrypt all stored travel plans to safeguard them from adversaries.

(A third valid NFR: "The system shall have a user-friendly interface to ensure ease of use.")

Format check. Full sentences, numbered, two labelled headings, and the **specific user** named ("travelers", not "user"). This is where the marks are.

Q2 — LLM Songwriting SaaS

6 marks

A(i). Is the Waterfall model suitable? 1

No, the Waterfall model is not suitable for this scenario. Waterfall requires that requirements are well defined and reasonably stable, it has **no feedback path**, it is **difficult to incorporate requirement changes**, and a **working product is only available after the last step**. Here the team explicitly wants to **deploy some features now and implement others in future** (instrumentals, earning options, collaboration, Apple Music), which Waterfall cannot support.

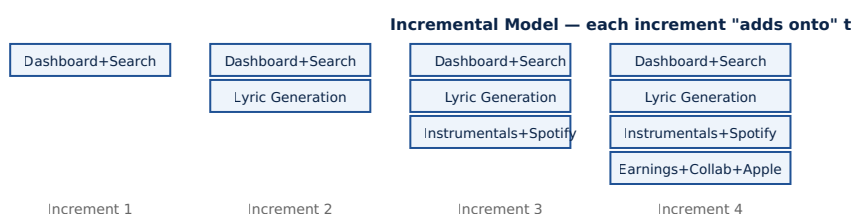
A(ii). Which SDLC model would be suitable? 1

The Incremental Model.

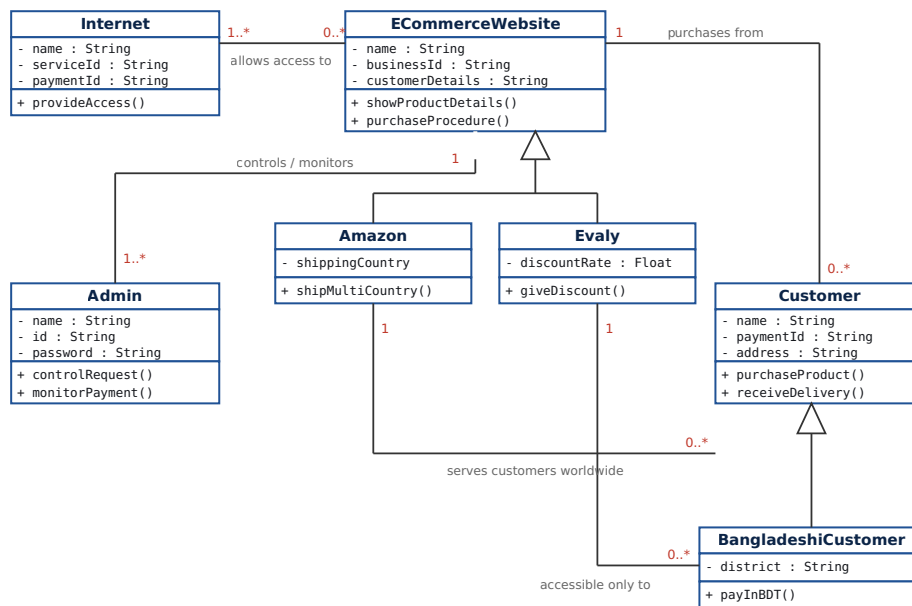
A(iii). Explain the chosen SDLC model 4

*The Incremental Model is a software process model where the software is **delivered in increments**.*

- The customer wants to use the software **from the very beginning** of the project — matching the friends' wish to launch and deploy features now.
- The customer is **quite well-known about the requirements**. The SRS is done and **locked at the start of each phase** until development of that phase ends. Here the feature list is already clearly specified: dashboard, search songs, filter by genre, view creation prompts, song generation page, lyric generation, regenerate lyrics.
- The software is divided into a **fixed number of increments** to be delivered to the customer — the scenario states they will **deploy it in four parts**.
- The model **"adds onto"** new sections as increments. For example: increment 1 delivers the dashboard with search and genre filter; increment 2 adds the song generation and lyric regeneration page; increment 3 adds instrumentals and Spotify integration; increment 4 adds earning options, collaborative features and Apple Music integration.
- It is an **Evolutionary Process Model**, so requirements can change and the team can return to previous phases. Its advantage is **customer involvement** — feedback during development makes the product correct, efficient and reliable (*building the right product*). Its disadvantage is the **risk of the project expanding beyond its initial scope**.
- Increments are **prioritised by customers**, so there is a better chance of success, and parallel stages such as requirement collection and planning can take place while construction and development continue.



Q3 — E-Commerce Class Diagram 5 marks



Class diagram for the e-commerce scenario.

Justification of each element (write 3-4 lines of this beside the diagram)

Element	Evidence from the scenario
Internet class	"All of our internet providers have names, service IDs, and payment IDs. " Associated with ECommerceWebsite because "the Internet allows us to use online sites."
ECommerceWebsite class	"E-Commerce websites consist of product details and purchase procedures " → operations. Amazon and Evaly each "has a name, business ID, and customer details " and each is "the generalized form of an e-commerce website" — so these three attributes belong to the superclass.
Amazon and Evaly	Generalization (hollow triangle pointing at ECommerceWebsite) — the scenario explicitly says both are "the generalized form of an e-commerce website". Amazon adds "shipping between multiple countries"; Evaly adds "attractive discounts on purchases".
Admin class	"Admin has a name, id, and password. It controls customer requests and monitors payment status. " Associated with ECommerceWebsite ("Like other websites, E-Commerce websites also need Admin").
Customer class	"The customer has a name, paymentId, Address " and "customers purchase products and receive online delivery."
BangladeshiCustomer	Generalization from Customer — "Customers can be from different parts of the world. Like Bangladeshi Customers. " Linked to Evaly because "Customers of Bangladesh can only access this website."

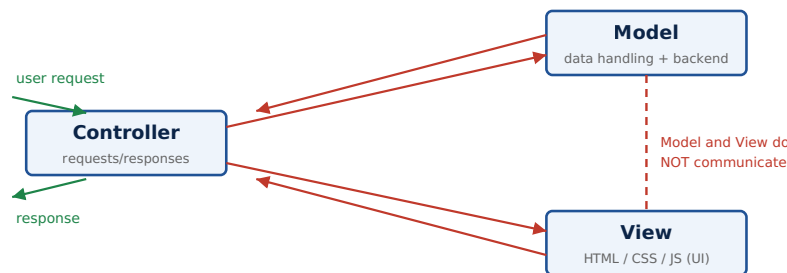
The mark-winning detail here is the two generalization hierarchies. The words "generalized form" and "like Bangladeshi Customers" are the question's keywords — if you draw everything as plain associations you lose most of the 5 marks. Also connect **Evaly → BangladeshiCustomer** specifically (not to Customer), because the scenario restricts access to Bangladesh only.

Q4 — "Bracu Shop" Architecture 7 marks

A. Which software architecture is suitable? 1

MVC (Model-View-Controller) Architecture.

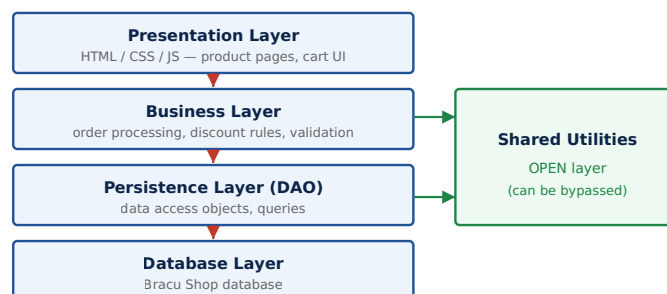
It matches the scenario exactly: MVC **separates presentation and interaction from the data-handling logic** and provides an extensible **separation of concern (SOC)**. The HTML/CSS/JS for the UI is the **View**; the data-handling and backend code is the **Model**; the separate section that users interact with and that handles user requests and responses is the **Controller**. Crucially, the scenario says the view section and the data section "**will not communicate with each other**" — in MVC the Controller communicates with both Model and View, which is what keeps them apart.



B. Can we choose any other architecture? Explain with diagram. 4

Yes — the Layered Architecture can also be used, because it likewise enforces separation of concern.

*Layered architecture organises the system into a set of layers (or abstract machines), each of which provides a set of services. It supports the incremental development of sub-systems in different layers — when a layer interface changes, only the **adjacent layer** is affected.*



CLOSED layer = accessed only by the layer directly above → isolation.

- Each layer has one responsibility, so the UI code (Presentation) and the data code (Persistence/ Database) are fully separated — exactly the separation of concern the scenario demands.
- **Closed layer** — can only be accessed by the layer above it; a change in one layer does not affect others, providing **isolation**. **Open layer** — can be bypassed by upper layers, e.g. a shared-utilities layer. Too many open layers ruins the essence of the architecture.
- **Advantages:** easy replacement or addition of entire layers as long as the interface is maintained; testing is easy because components are isolated; parallel team work.
- **Disadvantages:** a clean separation between layers is often difficult in practice — a high-level layer may have to interact directly with lower-level layers; performance suffers from multiple levels of interpretation of a service request; parallel processing is not possible because the structure is sequential.

C. Git — stage, commit, push to Dev1Branch 2

```
git add .
git commit -m "first commit done"
git push origin Dev1Branch
```

Command	What it does
<code>git add .</code>	Stages all changed files in the current directory (and sub-directories) into the staging area. <code>git add -A</code> is equivalent; <code>git add filename</code> stages a single file.
<code>git commit -m "first commit done"</code>	Commits the staged changes to the local repository with the message <i>first commit done</i> .
<code>git push origin Dev1Branch</code>	Pushes the committed code to the branch named Dev1Branch on the remote repository origin.

If *Dev1Branch* does not exist locally yet, you would first run `git checkout -b Dev1Branch` (or `git switch -c Dev1Branch`), then push with `git push -u origin Dev1Branch`.

Mark allocation check: Q1 (1+4+2) = 7 · Q2 (1+1+4) = 6 · Q3 = 5 · Q4 (1+4+2) = 7 → **Total 25.**